

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Лабораторная работа №3 «Контейнеризация приложения
средствами Docker»**

Выполнила:

Клапнева Диана

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

Необходимо сделать отчёт по шаблону

Ход работы

Шаг 1: Напишем Dockerfile для каждого сервиса. Их структура будет почти одинаковой, но каждый сервис будет на своем порту (api-gateway - , user-service - 3001, estate-service - 3002, deal-service - 3003, messaging-service - 3004).

Выбран базовый образ (шаблон, содержащий все необходимое для запуска нашего приложения – среду окружения/библиотеки/конфигурационные файлы) node:18-alpine. Копируется файл package.json, устанавливаются все зависимости, даоое копируется остальной код. Собирается наш TypeScript проект, открывается нужный порт и запускается собранное приложение.

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install |
COPY . .
RUN npm run build
EXPOSE 3001
CMD ["node", "dist/index.js"]
```

Шаг 2:

Напишем .dockerignore, он одинаковый для всех сервисов.

```
node_modules
npm-debug.log
.env
.git
.gitignore
README.md
.vscode
.idea
coverage
.nyc_output
.DS_Store
*.log
dist
```

Шаг 3: Пропишем docker-compose.yml в корневой папке проекта (у меня это просто папка репозитория в Github).

```

services:
  postgres:
    image: postgres:15-alpine
    container_name: real_estate_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: maindb
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-db.sql:/docker-entrypoint-initdb.d/init-db.sql
    networks:
      - real_estate_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  rabbitmq:
    image: rabbitmq:3-management
    container_name: real_estate_rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    ports:
      - "5672:5672"
      - "15672:15672"
    volumes:
      - rabbitmq_data:/var/lib/rabbitmq
    networks:
      - real_estate_network
    healthcheck:
      test: ["CMD", "rabbitmqctl", "status"]
      interval: 10s
      timeout: 5s
      retries: 5

    NODE_ENV: production
    POSTGRES_HOST: postgres
    POSTGRES_PORT: 5432
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: messages_db
    RABBITMQ_URL: amqp://rabbitmq:5672

    ports:
      - "3004:3004"
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
    networks:
      - real_estate_network
    restart: unless-stopped

  api-gateway:
    build: ./api-gateway
    container_name: api_gateway
    environment:
      PORT: 3000
      NODE_ENV: production
    ports:
      - "3000:3000"
    depends_on:
      - user-service
      - estate-service
      - deal-service
      - messaging-service
    networks:
      - real_estate_network
    restart: unless-stopped

networks:
  real_estate_network:
    driver: bridge

volumes:
  postgres_data:
  rabbitmq_data:

```

```

user-service:
  build: ./user-service
  container_name: user_service
  environment:
    PORT: 3001
    NODE_ENV: production
    POSTGRES_HOST: postgres
    POSTGRES_PORT: 5432
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: users_db
    RABBITMQ_URL: amqp://rabbitmq:5672
  ports:
    - "3001:3001"
  depends_on:
    postgres:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
  networks:
    - real_estate_network
  restart: unless-stopped

estate-service:
  build: ./estate-service
  container_name: estate_service
  environment:
    PORT: 3002
    NODE_ENV: production
    POSTGRES_HOST: postgres
    POSTGRES_PORT: 5432
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: estates_db
    RABBITMQ_URL: amqp://rabbitmq:5672
  ports:
    - "3002:3002"
  depends_on:
    postgres:
      condition: service_healthy
    rabbitmq:

```

Это главный управляющий проектом файл, тут перечислены все сущности, которые используются (мои 4 микросервиса и апи-шлюз, бд postgres и rabbitmq из ДЗ №4). Микросервисы запустятся только после health check бд и rabbitmq. В общей сети Real-estate-network все контейнеры видят друг друга по именам (не по ip). Также файл связывает порт внутри контейнера с портом моего пк, чтобы можно было зайти на <http://localhost:3000/> и попасть в апи-шлюз.

Также тут настраиваются тома volumes для хранения данных из postgresql, rabbitmq не терялись при перезапуске контейнера.

Шаг 3:

Запустим наши контейнеры. Вначале сборка длительная, но далее контейнеры будут пересобираться быстрее.

Командная строка

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\local>cd Documents\Github

C:\Users\local\Documents\Github>docker-compose up -d
[+] Running 7/7
  Container real_estate_rabbitmq Healthy
  Container real_estate_postgres Healthy
  Container user_service Started
  Container estate_service Started
  Container messaging_service Started
  Container deal_service Started
  Container api_gateway Running

C:\Users\local\Documents\Github>
```

Вот все контейнеры видны в docker desktop.

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	github	-	-	-	0.31%	10 minutes ago	☒ ⋮
☐	user_service	6c5809f89c1c	github-user-service	3001:3001	0%	10 minutes ago	☒ ⋮
☐	api_gateway	66722ee0f4ca	github-api-gateway	3000:3000	0%	19 minutes ago	☒ ⋮
☐	real_estate_postgres	4b7b767de7ec	postgres:15-alpine	5432:5432	0%	11 minutes ago	☒ ⋮
☐	real_estate_rabbitmq	f03b784c7163	rabbitmq:3-management	15672:15672 Show all ports (2)	0.31%	11 minutes ago	☒ ⋮
☐	messaging_service	0bce900bc072	github-messaging-service	3004:3004	0%	11 minutes ago	☒ ⋮
☐	estate_service	89fa8d4c1712	github-estate-service	3002:3002	0%	11 minutes ago	☒ ⋮
☐	deal_service	faec00f861b5	github-deal-service	3003:3003	0%	11 minutes ago	☒ ⋮

Вывод

С помощью данной работы я поняла необходимость докеризации приложения – это создает изоляцию сервисов в отдельных контейнерах, при этом вся система поднимается одной командой. Формируется единая сеть, в которой контейнеры общаются через простые имена. Любой пользователь сможет развернуть проект на своем устройстве. Теперь у меня есть практической реализации контейнеризации в docker.